KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Personal contact directory with search and update features

Submitted by

NIHA.S

25BCS242

Under the Guidance of

DHARANEESH

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled "**Personal contact directory with search and update features**" submitted in partial fulfillment of the requirements for the course [B.Sc COMPUTER SCIENCE**]** is a **record of my original work** carried out under the guidance of **[DHARANEESH]**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

NIHA.S

(Your Name)

CERTIFICATE

This is to certify that the project entitled "**Personal contact directory with search and update features**" is a **bona fide work** carried out by **[NIHA.S] [B.Sc]**, [COMPUTER

SCIENCE**]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course JAVA PROGRAMMING.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: COIMBATORE

Date:APRIL 2026

Submitted for the Viva-Voce Examination held on 10.04.2026

Guide: **Head of the Department**(Technical Trainer, KGCAS) (Technical Head, KGCAS)

DHARANEESH BOOPALAN

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	

4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The Personal Contact Directory with Search and Update Features is a software application developed to manage personal contact information in a structured and efficient manner. In traditional systems, contact details are stored manually using notebooks, diaries, or unorganized digital files, which often leads to problems such as data loss, duplication, and difficulty in searching information. These issues become more serious when the number of contacts increases.

This project aims to provide a computerized solution using Java programming language to store, retrieve, and manage contact details effectively. The system allows users to add new contacts, view stored contacts, search for specific contacts, update existing records, and delete unwanted entries. The data is maintained in an organized format using object-oriented programming concepts.

The system is developed using core Java concepts such as classes, objects, methods, loops, conditional statements, and ArrayList for dynamic data storage. The application is designed as a menu-driven program, making it simple and easy to use.

The main objective of this project is to reduce manual effort, improve accuracy, and provide a reliable method for managing personal contact information. This system also serves as a basic model for developing advanced applications with database integration and graphical interfaces.

1. INTRODUCTION

In today's digital world, communication plays a vital role in personal and professional life. People maintain a large number of contacts, including friends, relatives, colleagues, and business associates. Managing these contacts efficiently is very important for effective communication.

Traditionally, contact details are stored in notebooks or diaries. This method is not efficient, especially when the number of contacts increases. It becomes difficult to search for specific information, update existing records, or maintain proper organization. There is also a high risk of losing important data due to damage or misplacement.

To overcome these problems, computerized systems are widely used. A Personal Contact Directory System provides a digital platform for storing and managing contact information in an organized way. It allows users to perform operations such as adding, viewing, searching, updating, and deleting contacts easily.

This project is developed using Java programming language and demonstrates how software can be used to solve real-world problems. The system is simple, efficient, and user-friendly, making it suitable for beginners as well as practical use.

1.1 Objective

The primary objective of this project is to develop a simple and efficient system for managing personal contacts using Java.

The detailed objectives are:

- To create a digital system for storing contact details
- To provide easy access to stored information
- To implement search functionality for quick retrieval
- To allow updating of contact information
- To enable deletion of unwanted records

To reduce manual effort and human errors
To demonstrate object-oriented programming concepts

1.2 Overview

The Personal Contact Directory System is a menu-driven application developed using Java. It provides a simple interface where users can perform various operations related to contact management.

Users can enter contact details such as name, phone number, and email address. The system stores this information and allows users to manage it efficiently. The application includes multiple functionalities such as adding new contacts, viewing existing contacts, searching for specific contacts, updating records, and deleting contacts.

The system is designed in such a way that it can handle multiple contacts effectively. It uses data structures like ArrayList to store data dynamically. This project demonstrates how programming can be applied to real-life scenarios and provides a foundation for developing more advanced applications.

1.3 Features

The system includes the following features:

- Add Contact
  Allows users to enter new contact details such as name, phone number, and email
- View Contacts
  Displays all stored contact details in an organized format
- Search Contact
  Enables users to search for a contact using the name
- Update Contact
  Allows modification of existing contact details
- Delete Contact
  Removes unwanted or outdated contact information
- User-Friendly Interface
  Provides a simple menu-driven interface for easy usage
- Efficiency
  Reduces time and effort compared to manual systems

2.PROGRAM ANALYSIS

Program analysis is an important phase in software development where the problem is studied in detail and the requirements of the system are clearly identified. In this phase, the existing system is analyzed to understand its drawbacks, and a suitable solution is proposed to overcome those limitations. Proper analysis helps in designing an efficient and reliable system.

2.1 Problem Definition

In traditional contact management systems, people usually store contact details in notebooks, diaries, or unorganized digital files. This method is not efficient and leads to several problems. As the number of contacts increases, it becomes very difficult to manage and retrieve information quickly.

One of the major problems in manual systems is the difficulty in searching for specific contact details. Users need to go through multiple pages or records to find a particular contact, which is time-consuming and inconvenient. In addition, there is a high possibility of data duplication, where the same contact may be recorded more than once.

Another significant issue is the risk of data loss. Physical records such as notebooks can be easily damaged, misplaced, or lost. Even digital files without proper structure can lead to accidental deletion or corruption of data. Updating contact details in manual systems is also difficult, as it requires rewriting or modifying records manually.

Furthermore, manual systems lack proper organization and structure, making it difficult to maintain large volumes of data. These limitations reduce efficiency and increase the chances of errors.

Therefore, there is a need for a computerized system that can store, manage, and retrieve contact information efficiently. The proposed system aims to overcome these problems by providing a structured and automated solution.

2.2 Project Overview

The Personal Contact Directory with Search and Update Features is designed to provide a simple and effective solution for managing contact information. The system is developed using Java programming language and follows a menu-driven approach for easy user interaction.

The application allows users to store contact details such as name, phone number, and email address in a structured format. The stored data can be easily accessed, modified, or deleted whenever required. The system ensures that all records are maintained in an organized manner.

The main functionalities of the system include adding new contacts, viewing all stored contacts, searching for specific contacts, updating existing records, and deleting unwanted entries. Each functionality is implemented as a separate module to ensure clarity and efficiency.

The system uses object-oriented programming concepts such as classes and objects to represent contact data. The use of ArrayList allows dynamic storage of data, enabling the system to handle multiple contacts efficiently.

Overall, the project demonstrates how a simple software application can solve real-world problems by automating manual tasks and improving data management.

2.3 Scopes and Limitations

Scope

The scope of the Personal Contact Directory System includes managing personal contact information in a digital format. The system is designed to handle basic operations such as storing, retrieving, updating, and deleting contact details.

It can be used for personal purposes where individuals need to maintain a list of contacts in an organized manner. The system can also be applied in small offices or institutions where basic contact management is required.

The project serves as a foundation for developing more advanced applications. It can be extended to include additional features such as database integration, graphical user interface, and online access.

Limitations

Despite its advantages, the system has certain limitations. It is a console-based application and does not provide a graphical user interface, which may limit user experience. The data is stored temporarily in memory and is not saved permanently, which means all data will be lost once the program is terminated.

The system also supports only basic functionalities and does not include advanced features such as sorting, filtering, or multi-user access. Additionally, it may not be suitable for handling very large datasets.

These limitations can be overcome in future enhancements by integrating a database, developing a GUI-based application, and adding more advanced features.

3.SYSTEM DESIGN

System design is the process of planning the structure and components of the system. It acts as a blueprint for developing the application and ensures that all parts of the system work together effectively.

3.1 Modules

The Personal Contact Directory System is divided into several modules, each responsible for performing a specific function. This modular approach makes the system easy to understand, develop, and maintain.

1. Add Contact Module

This module allows the user to enter new contact details such as name, phone number, and email address. The entered data is stored in the system using objects. This module ensures that all required information is collected and stored correctly.

2. View Contact Module

The view module displays all the stored contacts in a clear and organized format. It allows users to see the complete list of contacts along with their details. This helps in verifying stored data and maintaining records.

3. Search Module

The search module enables users to find a specific contact by entering the name. The system compares the input with stored data and displays the matching record. This module improves efficiency by reducing the time required to locate information.

4. Update Module

This module allows users to modify existing contact details. If there are any changes in phone number or email address, the user can update the information without deleting the entire record. This ensures that the data remains accurate and up-to-date.

5. Delete Module

The delete module is used to remove unwanted or outdated contacts from the system. This helps in maintaining clean and organized data. It also prevents duplication and unnecessary storage usage.

6. Exit Module

This module allows the user to safely exit the application. It ensures that the program terminates properly without affecting the system.

4.METHODOLOGY

Methodology refers to the systematic and organized approach followed to design, develop, and implement the Personal Contact Directory with Search and Update Features. It explains how the problem is analyzed and how

the solution is developed step by step. A proper methodology ensures that the system is reliable, efficient, and meets all user requirements.

The development of this project follows several important phases such as problem analysis, system design, program development, testing and validation, and documentation. Each phase plays a significant role in building the complete system.

4.1. Problem Analysis

Problem analysis is the first and most important step in the development process. In this phase, the existing manual system of maintaining contact details is carefully studied to identify its drawbacks and limitations.

In traditional methods, contact details are stored in notebooks or simple files, which makes it difficult to manage data efficiently. Some of the common problems identified are difficulty in searching contacts, duplication of records, risk of data loss, and time-consuming processes. Updating or deleting information in manual systems is also not easy and may lead to inconsistencies.

After identifying these issues, the need for a computerized system is recognized. The proposed system should be able to store contact details in a structured format, provide quick search functionality, and allow easy updating and deletion of records. This phase helps in clearly defining the requirements of the system.

4.2. System Design

System design is the process of planning how the system will be structured and how different components will interact with each other. It acts as a blueprint for developing the application.

In this project, the system is designed as a menu-driven application to ensure simplicity and ease of use. The design includes dividing the system into different modules such as add contact, view contact, search contact, update contact, and delete contact.

Data is stored using objects, where each contact is represented as an object containing attributes like name, phone number, and email address. An ArrayList is used to store multiple contacts dynamically. The design also includes defining the input and output operations and the flow of the program.

A well-planned system design reduces complexity and helps in smooth implementation and maintenance of the application.

4.3. Program Development

Program development is the phase where the actual coding of the system is carried out. Based on the system design, the program is implemented using Java programming language.

In this phase, classes and objects are created to represent contact details. Variables are defined to store information such as name, phone number, and email address. Methods are written for different operations like adding, viewing, searching, updating, and deleting contacts.

Loops are used to allow repeated execution of the menu, and conditional statements are used to control the flow of the program based on user input. The Scanner class is used to accept input from the user.

The program is developed using core Java concepts such as object-oriented programming, modular programming, and data structures. Proper coding practices are followed to ensure readability, maintainability, and efficiency of the program.

4.4. Testing and Validation

Testing is a crucial phase in software development that ensures the system works correctly and meets the required specifications. In this phase, the developed program is tested with different inputs to identify errors or bugs.

Various types of testing are performed to verify the functionality of the system. Unit testing is used to test individual modules separately, while system testing is used to test the entire application as a whole.

The system is tested for different scenarios such as adding multiple contacts, searching for existing and non-existing contacts, updating records, and deleting contacts. Validation ensures that the system produces accurate and expected results.

Proper testing improves the reliability and performance of the system and ensures that it functions without errors.

4.5. Result and Documentation

After successful testing, the results of the system are analyzed and documented. The system is able to perform all operations such as adding, viewing, searching, updating, and deleting contacts effectively.

The documentation includes details about the system design, methodology, algorithm, and outputs. It helps users understand the working of the system and serves as a reference for future improvements.

Proper documentation is important for maintaining the system and for making any enhancements in the future.

4.6. Algorithm

An algorithm is a step-by-step procedure used to solve a problem. It provides a clear logic for the execution of the program.

Algorithm for Personal Contact Directory:

1.Start the program
2.Display menu options
3.Accept user choice
4.If user selects add contact, enter and store contact details
5.If user selects view contacts, display all stored records
6.If user selects search contact, find and display matching record
7.If user selects update contact, modify existing details
8.If user selects delete contact, remove the record
9.Repeat the process until the user chooses to exit
10.Stop the program

4.7. Flowchart

Start → Display Menu → Enter Choice → Perform Operation (Add/View/Search/Update/Delete) → Display Result → Repeat → End

Students can draw using:

- draw.io
- Lucidchart
- Microsoft Visio
- MS Word Shapes
- Pencil and Scale (for record work)

5.SOURCE CODE

A class named Contact is created to store the contact details such as name, phone number, and email address. The main class contains a menu-driven program that allows the user to perform operations like adding, viewing, searching, updating, and deleting contacts.

The system uses an ArrayList to store multiple contact records dynamically. The Scanner class is used to get input from the user. The program uses loops and conditional statements to perform different operations based on user choice.

The following inputs are provided to the Personal Contact Directory System:

- User Choice:

The user selects an option from the menu such as Add Contact, View Contacts, Search Contact, Update Contact, Delete Contact, or Exit.

- Name:

The user enters the name of the contact as text.

- Phone Number:

The user enters the phone number of the contact.

- Email Address:

The user enters the email ID of the contact.

- Search Name:

The user enters the name of the contact to search for specific details.

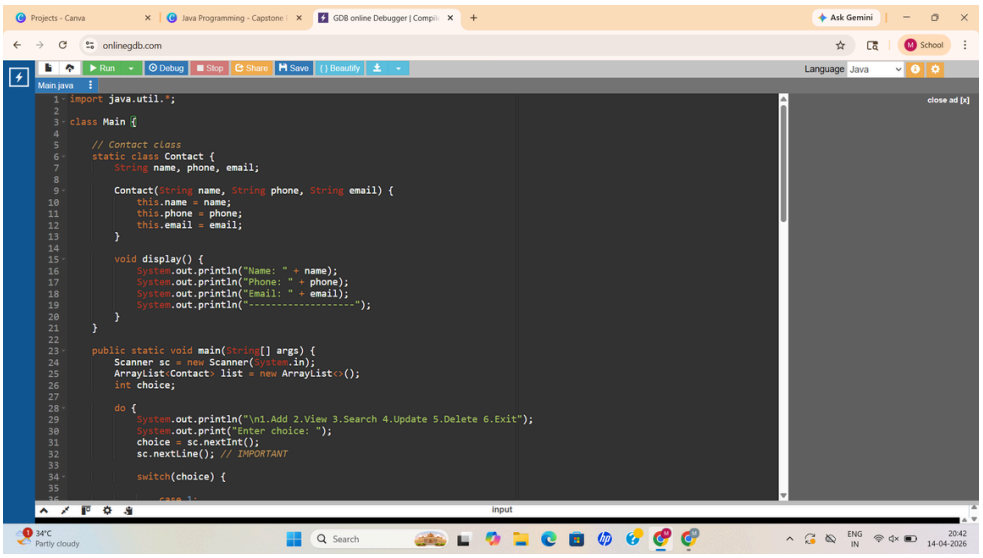
- Update Details:

The user provides new phone number and email to update an existing contact.

- Delete Name:

The user enters the name of the contact to be deleted from the system.

6.INPUT AND OUTPUT SCREENS



```java
import java.util.*;

class Main {

    // Contact class
    static class Contact {
        String name, phone, email;

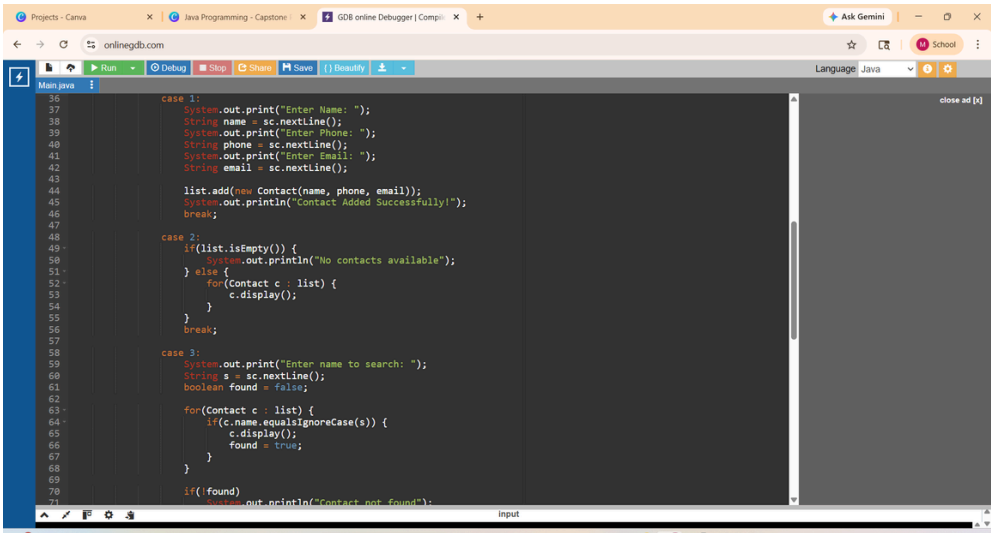
        Contact(String name, String phone, String email) {
            this.name = name;
            this.phone = phone;
            this.email = email;
        }

        void display() {
            System.out.println("Name: " + name);
            System.out.println("Phone: " + phone);
            System.out.println("Email: " + email);
            System.out.println("--------------------");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Contact> list = new ArrayList<>();
        int choice;

        do {
            System.out.println("\n1.Add 2.View 3.Search 4.Update 5.Delete 6.Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();
            sc.nextLine(); // IMPORTANT

            switch(choice) {
```



```java
            case 1:
                System.out.print("Enter Name: ");
                String name = sc.nextLine();
                System.out.print("Enter Phone: ");
                String phone = sc.nextLine();
                System.out.print("Enter Email: ");
                String email = sc.nextLine();

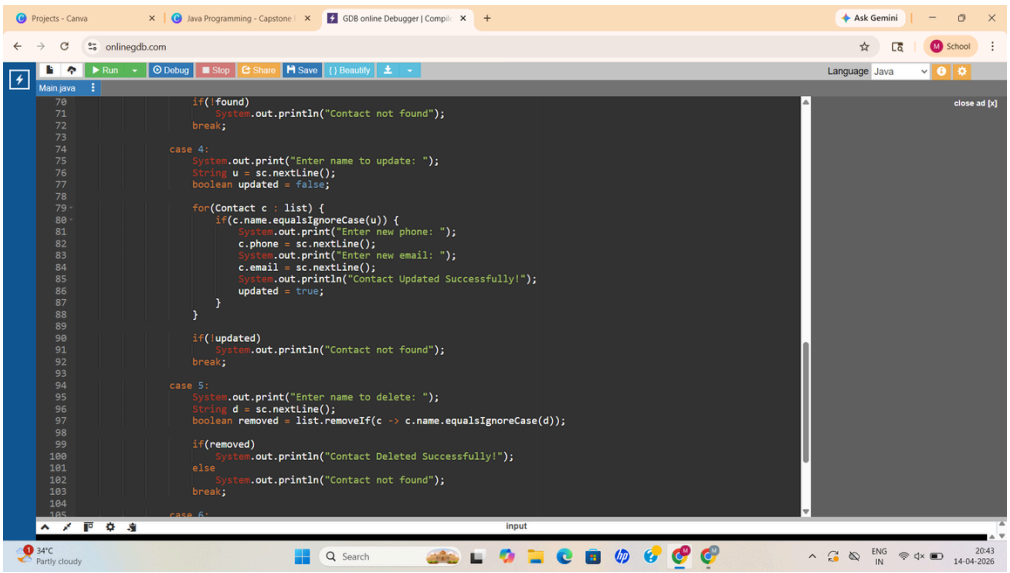
                list.add(new Contact(name, phone, email));
                System.out.println("Contact Added Successfully!");
                break;

            case 2:
                if(list.isEmpty()) {
                    System.out.println("No contacts available");
                } else {
                    for(Contact c : list) {
                        c.display();
                    }
                }
                break;

            case 3:
                System.out.print("Enter name to search: ");
                String s = sc.nextLine();
                boolean found = false;

                for(Contact c : list) {
                    if(c.name.equalsIgnoreCase(s)) {
                        c.display();
                        found = true;
                    }
                }

                if(!found)
                    System.out.println("Contact not found");
```



```java
                if(!found)
                    System.out.println("Contact not found");
                break;

            case 4:
                System.out.print("Enter name to update: ");
                String u = sc.nextLine();
                boolean updated = false;

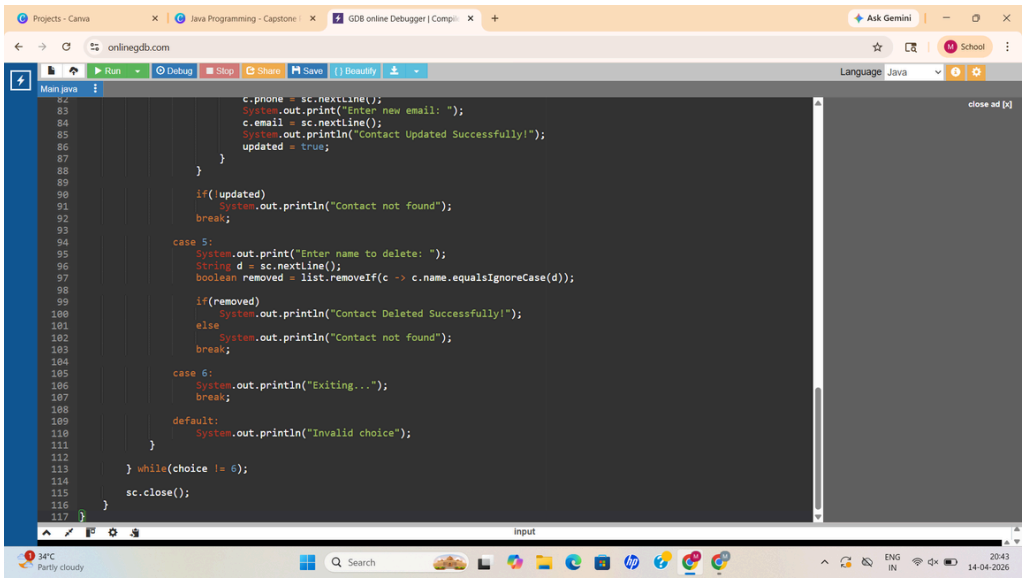
                for(Contact c : list) {
                    if(c.name.equalsIgnoreCase(u)) {
                        System.out.print("Enter new phone: ");
                        c.phone = sc.nextLine();
                        System.out.print("Enter new email: ");
                        c.email = sc.nextLine();
                        System.out.println("Contact Updated Successfully!");
                        updated = true;
                    }
                }

                if(!updated)
                    System.out.println("Contact not found");
                break;

            case 5:
                System.out.print("Enter name to delete: ");
                String d = sc.nextLine();
                boolean removed = list.removeIf(c -> c.name.equalsIgnoreCase(d));

                if(removed)
                    System.out.println("Contact Deleted Successfully!");
                else
                    System.out.println("Contact not found");
                break;

            case 6:
```



```
                        c.phone = sc.nextLine();
                        System.out.print("Enter new email: ");
                        c.email = sc.nextLine();
                        System.out.println("Contact Updated Successfully!");
                        updated = true;
                    }
                }

                if(!updated)
                    System.out.println("Contact not found");
                break;

            case 5:
                System.out.print("Enter name to delete: ");
                String d = sc.nextLine();
                boolean removed = list.removeIf(c -> c.name.equalsIgnoreCase(d));

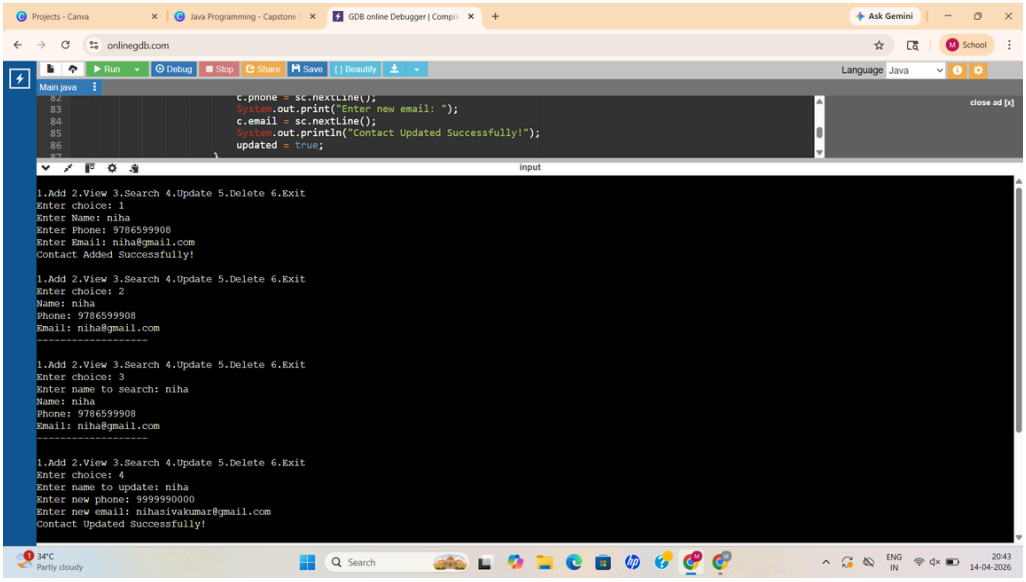
                if(removed)
                    System.out.println("Contact Deleted Successfully!");
                else
                    System.out.println("Contact not found");
                break;

            case 6:
                System.out.println("Exiting...");
                break;

            default:
                System.out.println("Invalid choice");
        }

    } while(choice != 6);

    sc.close();
}
}
```

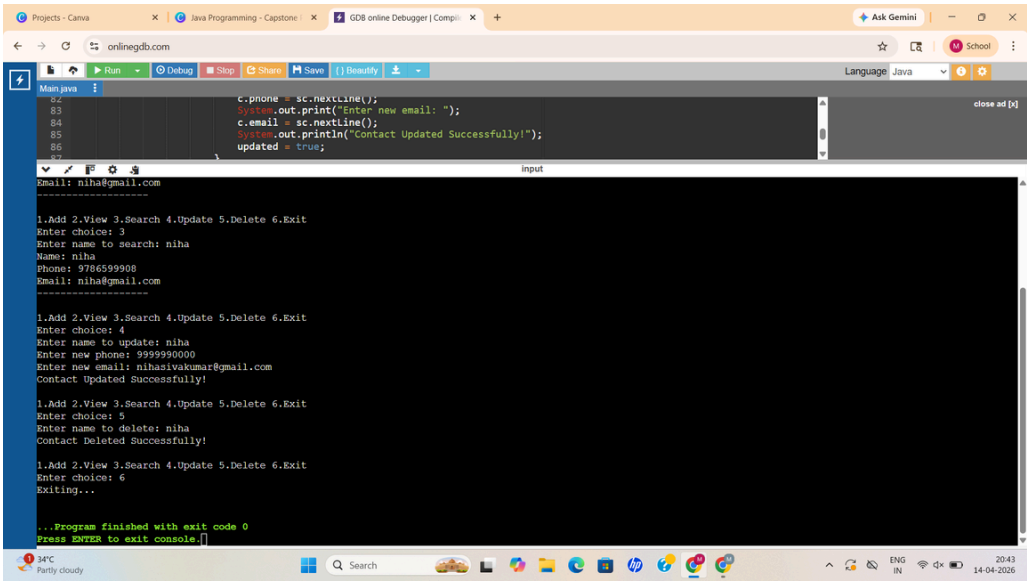


```
1.Add 2.View 3.Search 4.Update 5.Delete 6.Exit
Enter choice: 1
Enter Name: niha
Enter Phone: 9786599908
Enter Email: niha@gmail.com
Contact Added Successfully!

1.Add 2.View 3.Search 4.Update 5.Delete 6.Exit
Enter choice: 2
Name: niha
Phone: 9786599908
Email: niha@gmail.com
--------------------

1.Add 2.View 3.Search 4.Update 5.Delete 6.Exit
Enter choice: 3
Enter name to search: niha
Name: niha
Phone: 9786599908
Email: niha@gmail.com
--------------------

1.Add 2.View 3.Search 4.Update 5.Delete 6.Exit
Enter choice: 4
Enter name to update: niha
Enter new phone: 9999990000
Enter new email: nihasivakumar@gmail.com
Contact Updated Successfully!
```



```
Email: niha@gmail.com
--------------------

1.Add 2.View 3.Search 4.Update 5.Delete 6.Exit
Enter choice: 3
Enter name to search: niha
Name: niha
Phone: 9786599908
Email: niha@gmail.com
--------------------

1.Add 2.View 3.Search 4.Update 5.Delete 6.Exit
Enter choice: 4
Enter name to update: niha
Enter new phone: 9999990000
Enter new email: nihasivakumar@gmail.com
Contact Updated Successfully!

1.Add 2.View 3.Search 4.Update 5.Delete 6.Exit
Enter choice: 5
Enter name to delete: niha
Contact Deleted Successfully!

1.Add 2.View 3.Search 4.Update 5.Delete 6.Exit
Enter choice: 6
Exiting...


...Program finished with exit code 0
Press ENTER to exit console.
```

7.DISCUSSION & CONCLUSION

7.1 Discussion

The Personal Contact Directory system was developed to simplify the management of contact details. It replaces manual methods like notebooks, which are time-consuming and difficult to maintain.

The system is menu-driven and allows users to add, view, search, update, and delete contacts easily. The search feature helps in quickly finding contacts, while the update option allows modification without re-entering data.

The use of ArrayList enables dynamic storage, and Java OOP concepts help in organizing the program effectively. The system works correctly for all basic operations.

However, it has some limitations such as no data storage after program exit, lack of input validation, and no graphical interface.

7.2 Conclusion

The project successfully provides a simple solution for managing contacts using Java. It reduces manual work and improves efficiency.

It demonstrates important programming concepts like classes, objects, loops, and ArrayList. The system can be further improved by adding database support, GUI, and validation features.

8.REFERENCES

Books

- Herbert Schildt – *Java: The Complete Reference*
- E. Balagurusamy – *Programming with Java*

Websites

- https://www.oracle.com/java
- https://www.geeksforgeeks.org
- https://www.w3schools.com
- https://www.javatpoint.com